Ders 26 Çalışma Özeti

Ders 26 Çalışma Özeti

Genel Konular

- Dönemin son dersi (11.06.2021). Önceki derste yarım kalan örnekler tamamlanır:
  - Sepet analizinde birlikte satılan ürün üçlüleri (üç boyutlu matris) ve analizi.
  - Olasılıksal bir algoritma: kumarbaz problemi (gambler's ruin) simülasyonu.
  - Görüntüde tuz-biber gürültüsünü temizlemek için ortanca (median) filtresi.
  - Resmi küçültme ve büyütme.
- Sepet analizi tekrarı
  - Ürünler marka veya alt tür ayrımı yapılmadan grup olarak düşünülür (domates domatestir, kalem kalemdir).
  - (sepet, ürün) listesinden 0-1 bir `A` matrisi (ürün × sepet), ondan da ürün çiftlerinin birlikte satış sayılarını tutan bir `T` matrisi elde ediliyordu.
- Ürün üçlüleri
  - Bir ürün bir sepette bulunduysa, aynı sepette ondan sonraki ürünlere, bulunan ikinci üründen sonra da üçüncü ürünlere bakılır. Üç ürünün kesişimi bir yükseklik demektir, bu yüzden üç boyutlu bir `T[i][j][k]` matrisi gerekir. Bu dönem ilk kez üç boyutlu matrisle çalışıldı.
  - Kod (`Sepet3D.c`): `i` ürünler, `j` `i+1`'den, `k` `j+1`'den ürünler, `m` sepetler üzerinde döner. `T[i][j][k] = T[i][j][k] + A[i][m] * A[j][m] * A[k][m];`. Üçü de 1 ise çarpım 1 olur ve sayaç artar.
  - Ürünlerin bulunduğu sepetleri tek döngüyle doldurmak yeterlidir. İki sütun için ayrı bir iç döngü yazmak yanlış değildir, ama döngünün işlemcide ek yükü vardır.
  - Derste örnek veride önce yalnızca (1, 2, 4) üçlüsü bulundu. 1. sepete 3 eklenince (2, 3, 4) üçlüsü de göründü.
  - Üçlüler için karmaşıklık n^4'tür (üç ürün döngüsü ve sepet döngüsü). En çok birlikte satılan 10'lu grubu bu yöntemle aramak n^{11}'e çıkar, yani çok kötü bir yöne gider. Veri madenciliğinde bunun için Apriori gibi algoritmalar vardır.
- `if` ile çarpım arasında karmaşıklık ve süre
  - Temel işlem, her durumda kesin yapılan karşılaştırmadır. Sabit sayıdaki karşılaştırma ve toplamalar 1 sayılır.
  - İkili sürümde `i` ve `j` iç içe n'e kadar, `k` n'e kadar döner. İçteki toplamdan n^2 mertebesinde bir terim gelir, dış döngüyle n^3 olur. Bu bir üst sınırdır: 10 ürün ve 10 sepette karşılaştırma en fazla 10^3 kez yapılır. Algoritma bu çizginin altında bir yerde çalışır, onu aşmaz.
  - Bellek: başlangıçtaki $n \times 2$ liste, n^2'lik `A` ve n^2'lik `T`. Toplam ek bellek $2n^2$, yani n^2.
  - `if (A[k][i] == 1 && A[k][j] == 1)` yazılırsa ilk koşul yanlış olduğunda ikinci kontrol yapılmaz ve ekleme de yapılmaz. Çarpım (`T += A*A`) yazılırsa her durumda bütün işlemler yapılır.
  - Hocanın kaba sayımı: `if`'li sürümde kesin yapılan 5, en fazla 9 temel işlem; çarpımlı sürümde her zaman 7 işlem vardır. Atamalar dikkate alınırsa `if`'li sürüm daha kısa sürer. Ama bir karşılaştırmanın saniye cinsinden maliyeti işlemciden işlemciye değişir. Kesin sonuç için iki sürüm aynı bilgisayarda, başka program çalışmazken `clock` ile defalarca (50, 100 kez) ölçülüp ortalamaları karşılaştırılmalıdır.
  - Big-O bir üst sınır gösterimidir. Big-Omega ve Big-Theta gelecek dönemlerde görülecek.
- Kumarbaz problemi (olasılıksal algoritma)
  - Her oyunda p olasılıkla 1 TL kazanılır, $1 - p$ olasılıkla 1 TL kaybedilir. Elinde n TL

olan oyuncu, m TL kazanınca ($n + m$'ye ulaşınca) veya parası bitince oyunu bırakır. Oyuncunun kazanarak çıkma olasılığı nedir?
  - Tek oyuncu simülasyonu: `while (n < hedef && n > 0)` döngüsünde `rand() / (float) RAND_MAX` ile 0-1 arasında bir sayı üretilir. Sayı $1 - p$'den büyükse `n++`, değilse `n--`. Oyuncunun parası her adımda bir diziye yazılır ve grafiği çizilir.
  - Çok oyunculu sürüm: deney `K` kez tekrarlanır, hedefe ulaşanlar sayılır, sayı `K`'ya bölünür. `n` her denemenin başında ilk değerine döndürülmelidir.
  - `srand(time(NULL))` yalnızca bir kez, `main`'in başında çağrılmalıdır. Birçok fonksiyon varsa onların içinde tekrar çağrılmaz.
  - Derste $n = 100, m = 10, p = 0.473$ ile kazanma oranı 0.335 bulundu. Ders notundaki farklı denemelerin sonuçları 0.339-0.341 arasındadır.
  - Sonuç n'e bağlı değildir. Kazanma olasılığı yaklaşık $\left(\frac{p}{1-p}\right)^m$'dir ve bundan küçük kalır. Yani kasa her zaman kazanır. Bazı oyuncular kazanır, ama oynadıkça ortalamada kaybedilir.
  - Benzetme: yazı tura 10 kez atılınca 10'u da yazı gelebilir. 10 milyon kez atılınca oran 1/2'ye yaklaşır.
- Ortanca (median) filtresi
  - Önceki derste ortalama filtresinin gürültüyü de ortalamaya kattığı için başarılı olmadığı görülmüştü. Büyük pencere ise resmi bulanıklaştırıyordu.
  - Anıl'ın önerisi: 0 ve 255 olan pikselleri ortalamaya katmamak ve geri kalanların sayısına (sayaçla) bölmek. Bu, gürültüyü giderdi ama resmin kendi gerçek siyah ve beyaz bölgelerini de bozdu (saç, gölgeler). Resmin dokusunu bozmamak gerekir.
  - Selim'in önerisi: ortalamadan standart sapmanın belli bir katından uzak olan aykırı (outlier) pikselleri atmak. Hoca bunun olabileceğini, ama daha kolay bir yöntem olduğunu söyler.
  - Ortanca filtresi: penceredeki 9 piksel sıralanır. 0'lar başa, 255'ler sona gider. Ortadaki eleman (ortanca) alınır. Gürültü toplama hiç girmez. Photoshop'un tuz-biber gürültüsü filtresi de böyle çalışır.
  - Kod: pencere `filtre / 2` kadar içeriden başlar. Her piksel için pencere bir diziye kopyalanır, sıralanır (`bubble sort` benzeri basit bir sıralama) ve `dizi[(fw * fh) / 2]` yeni resme yazılır. Derste ilk denemede swap'ta yanlış indis yazıldığı için gürültü büyüdü, düzeltilince gürültü kayboldu.
  - Kenarlarda işlenmeyen bir şerit kalır. Programlar bunu resmin kenarlarına satır ve sütun ekleyerek (padding) çözer.
  - 9 × 9 pencere çok yavaşlar (iç içe dört döngü ve her pikselde 81 elemanlı sıralama) ve resmi bulanıklaştırır. Göz gibi ayrıntılı bölgelerde büyük pencere dokuyu bozar. Uygulamalarda genellikle 3 × 3 kullanılır.
- Resmi küçültme ve büyütme
  - Küçültme (shrink): `i` ve `j` ikişer artırılarak her iki pikselden biri yeni matrise (n/2 × m/2) yazılır. Yeni matris `malloc` ile dinamik ayrıldı. Bu konu gelecek dönem görülecek.
  - Büyütme (2n × 2m): ilk denemede her piksel yeni matriste `(m, n)` ve `(m+1, n+1)` hücrelerine kopyalandı. Sonuç karanlık ve satranç tahtası gibi bozuk bir görüntü oldu.
  - Daha iyi bir yöntem: yeni hücreye mevcut piksel ile sağdaki pikselin ortalamasını yazmak. Daha da iyisi sağdaki, alttaki ve çaprazdaki pikselle dördünün ortalamasını almak. Bu, aradaki değerlerin tahmin edildiği bir interpolasyondur.
  - En iyisi, resmin dokusuna bakarak (ör. 20 piksellik bir bölgeden) bir fonksiyon (kuadratik, kübik) üretip ara değerleri o fonksiyonla doldurmaktır. Bu sayısal analiz dersindeki interpolasyonla bağlantılıdır.
  - Ayna görüntüsü (mirror) de matris işlemiyle kolayca yapılır.
- Kapanış
  - Proje son teslim tarihi sınav takvimine göre güncellenecek. Bu derste projeden başka değerlendirme kalmadı. Projeler hemen kontrol edilmeye başlanacak.
  - Hoca, bu dersi ilk kez öğretim üyesi olarak verdiğini, öğretmenin kendisi için de bir öğrenme aracı olduğunu söyleyerek dönemi kapatır. Sayısal analiz için ertesi gün saat 9'da ve 14'te ayrı Zoom oturumları açılacak.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir algoritmanın karmaşıklığı bir üst sınırdır. Gerçek çalışma bu sınırın altında kalabilir. Döngüler ne kadar erken biterse o kadar iyidir.
- Süre ölçümleri tek başına belirleyici değildir. Aynı koşullarda defalarca ölçüp ortalamaya bakın.
- Olasılıksal simülasyonlarda deney sayısı arttıkça sonuç gerçek olasılığa yaklaşır.
- Gürültü giderirken resmin kendi dokusunu bozmayın. Tuz-biber gürültüsü için ortanca filtresi kullanın.

Kısa Tekrar Notları

- Üçlü birliktelik: `T[i][j][k] += A[i][m] * A[j][m] * A[k][m]`, karmaşıklık n^4.
- Kumarbaz problemi: kazanma olasılığı yaklaşık $\left(\frac{p}{1-p}\right)^m$, n'den bağımsız. $p = 0.473$, $m = 10$ için yaklaşık 0.34.
- Ortanca filtresi: pencereyi sırala, ortadakini al.
- Küçültme: ikişer atlayarak örnekleme. Büyütme: komşuların ortalamasıyla ara değer üretme.

Detaylı Açıklamalar

Formüllerin ve sonuçların kontrolü

Ders sonrasında Python ile kontrol edildi:

- Kumarbaz problemi için tam formül: $q = 1 - p$ olmak üzere $P(\text{kazanma}) = \frac{1-(q/p)^n}{1-(q/p)^{n+m}}$. $n = 100$, $m = 10$, $p = 0.473$ için 0.3392 çıktı. Yaklaşık formül $(p/q)^m$ de 0.3392 verdi. 20.000 denemelik bir simülasyonda oran 0.338 çıktı. Derste bulunan 0.335 ile uyumludur.
- Üçlü sepet analizi, 1. sepete 3 eklenmiş veriyle yalnızca (1, 2, 4) ve (2, 3, 4) üçlülerini birer kez buldu. Derste görülen sonuçla aynıdır.

Ders sonrası notlar

- **Kazanma olasılığının n'den bağımsız olması:** Yaklaşık formül $\left(\frac{p}{1-p}\right)^m$ yalnızca $p < 0.5$ ve elde yeterince para (n büyük) varken geçerlidir. Tam formülde n yer alır. Ama $q/p > 1$ olduğu için n büyüdükçe etkisi kaybolur. n küçükse (ör. 2 TL ile 10 TL kazanmaya çalışmak) kazanma olasılığı daha da düşüktür.
- **Büyütmede görüntünün kararmasının sebebi:** İlk denemede her piksel yalnızca `(m, n)` ve `(m+1, n+1)` hücrelerine yazıldı. `(m, n+1)` ve `(m+1, n)` hücrelerine hiçbir şey yazılmadı. Bu hücreler 0 (siyah) veya bellekte ne varsa o olarak kaldığı için görüntü kardardı ve satranç tahtası deseni oluştu. Bu bir interpolasyon sorunu değil, eksik doldurma sorunudur. Pikseli dört hücrenin hepsine kopyalamak (en yakın komşu, nearest neighbor) blok blok ama karanlık olmayan bir görüntü verir.
- **Ayna görüntüsü döndürme değildir:** Hoca ayna görüntüsü almanın matrisi 90 veya 60 derece döndürmek demek olduğunu söyledi. Ayna, matrisin her satırının ters çevrilmesidir (yatay yansıma). 90 derece döndürme ise transpoz ve ardından satırları ters çevirmektir (bkz. Ders 11 ve 12 notları). Piksel matrisinde 60 derece gibi açılar tam sayı indislerle yapılamaz, interpolasyon gerekir.
- **Ortanca için sıralama:** 9 elemanın ortancası için bütün diziyi sıralamak gerekmez. Ortadaki elemana kadar bir seçme sıralaması (selection sort) yeterlidir: 5 tur yeterli olur. Büyük pencerelerde bu fark hız kazandırır.
- **Küçültmede örtüşme (aliasing):** Her iki pikselden birini atmak, ince desenli bölgelerde titreşimli (moiré) görüntülere yol açabilir. Daha düzgün küçültme için önce 2 × 2'lik bloğun ortalaması alınır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.